

17 — Docker Containerization

Requirement: R21 (Docker Compose deployment)

Phase: O

What it is

When you build software on your laptop and then try to run it on a server, things break. Different Python version, missing system library, wrong database driver — the classic "works on my machine" problem. Docker solves this by packaging your application and *everything it needs* into a standardized, portable unit.

Here are the core concepts:

Container

A container is an isolated, lightweight package that bundles your app together with all its dependencies — the right Python version, the right libraries, the right system tools. It runs as if it has its own tiny operating system, but it shares the host machine's kernel, so it starts in seconds (unlike a full virtual machine which boots an entire OS).

Think of it this way: a virtual machine is like renting an entire apartment (own kitchen, own bathroom, own everything). A container is like renting a room in a shared house — you get your own private space, but you share the plumbing and electricity infrastructure. Much cheaper, much faster to move in.

Image

An image is the blueprint or recipe that containers are created from. It is a read-only snapshot: "Python 3.10, plus these pip packages, plus this application code, configured to run this command." You build an image once, and then you can create as many identical containers from it as you want.

The relationship is like a class vs. an instance in programming: the image is the class definition, and each running container is an instance of that class.

Dockerfile

A Dockerfile is the instruction file that tells Docker how to build an image, step by step. Each instruction creates a layer, and Docker caches layers so rebuilds are fast when only your code changes (but your dependencies haven't). The key instructions are:

Instruction	What it does
FROM	Start from a base image (e.g., python:3.10-slim)
WORKDIR	Set the working directory inside the container
COPY	Copy files from your machine into the image

RUN	Execute a command during the build (e.g., <code>pip install</code>)
EXPOSE	Document which port the app listens on
CMD	The default command to run when the container starts

Layer ordering matters for caching. You copy `requirements.txt` and install dependencies *before* copying your source code. That way, if only your code changes but dependencies haven't, Docker reuses the cached dependency layer instead of reinstalling everything. This can save minutes on every rebuild.

Multi-stage build

A multi-stage build uses two (or more) `FROM` instructions in one Dockerfile. The first stage does the heavy work (compiling, building, bundling), and the second stage copies only the finished output into a small, clean base image.

Why? Build tools are huge. Node.js with all your `node_modules` can be over 1 GB. But the final product — your compiled HTML/CSS/JS files — is just a few megabytes. A multi-stage build lets you use the big toolbox to build, then throw the toolbox away and ship only the finished product. The result is a final image of ~25 MB instead of ~1 GB.

docker-compose

When your project has multiple services that need to work together (a database, a backend API, a frontend, a data pipeline), starting each one individually with the right configuration would be tedious and error-prone. `docker-compose` lets you define all your services in one YAML file and start the entire stack with a single command: `docker compose up`.

It handles:

- **Networking** — all services automatically share a private network and can reach each other by service name (e.g., `postgres`, `search-api`)
- **Startup order** — `depends_on` with health checks ensures the database is ready before the API tries to connect
- **Environment variables** — centralized configuration via `env_file` and `environment`
- **Volumes** — persistent storage and file sharing between host and containers

Reverse proxy

A reverse proxy is a server that sits in front of your application and forwards incoming requests to the right backend service. In our project, `nginx` acts as a reverse proxy for the admin UI: it serves the static frontend files directly and forwards API requests (like `/auth/` and `/admin/`) to the FastAPI backend.

Why not just let the browser talk to FastAPI directly? In development, Vite's built-in proxy handles this. But in production, Vite is not running — you only have compiled static files. You need a real web server (`nginx`) to serve those files and proxy API calls to the backend.

Named volumes

By default, when you stop and remove a container, all its data disappears. Named volumes solve this by providing persistent storage that lives outside the container's filesystem. Even if you destroy and recreate the postgres container, the database files stored in the named volume survive.

Real-life analogy

Imagine you are in the international shipping business.

Before standardized shipping containers existed, every shipment was different. Dockworkers had to figure out how to load each oddly shaped crate onto a ship, then unload and reload everything onto a different truck. It was slow, fragile, and things broke constantly.

Standardized shipping containers changed everything. No matter what is inside — electronics, furniture, food — the container is always the same shape. Any crane can lift it. Any ship can carry it. Any truck can haul it. The contents are protected and isolated from everything else.

Docker containers work the same way:

- **A Docker image** is like the packing list and assembly instructions for loading a specific container
- **A Docker container** is the sealed, standardized box ready to ship — it runs identically on your laptop, your teammate's laptop, or a production server
- **A Dockerfile** is the detailed packing instructions that a warehouse worker follows to load the container
- **docker-compose** is the shipping manifest — it says "these 4 containers travel together on this ship, container A must be loaded before container B, and containers C and D need to be able to talk to each other through these specific doors"

Just like a shipping manifest coordinates multiple containers into one shipment, `docker-compose.yml` coordinates our postgres, ETL, search-api, and admin-ui containers into one deployable stack.

Why we used it here

This project (requirement R21) has four services that must work together as a unified stack:

1. postgres (pgvector) — the database

Uses the official `pgvector/pgvector:pg16` image — no Dockerfile needed, someone else already built and published it. We configure it with environment variables and attach a **named volume** (`pgdata`) so the database survives container restarts. The **healthcheck** is critical: it runs `pg_isready` every 5 seconds so other services know when postgres is actually accepting connections, not just when the container has started (the container starts before postgres finishes initializing).

2. etl — the data pipeline

A one-shot Python container. It runs the ETL pipeline once and then exits (it is not a long-running server). It uses `depends_on` with `condition: service_healthy` to wait for postgres to be fully ready. The `data/` directory is mounted as a volume so the ETL can read the source JSON files from the host without baking them into the image.

3. *search-api* — the FastAPI backend

The search API serves queries on port 8000. Its Dockerfile has an important detail: it copies *both* `search_api/` and `bilingual_etl/` into the image. Why? Because the search API imports code from `bilingual_etl` — specifically the embedding provider module. When the API needs to generate an embedding for a search query, it calls the same embedding provider that the ETL pipeline uses. Without copying `bilingual_etl/` into the search-api image, those imports would fail with `ModuleNotFoundError`.

4. *admin-ui* — the React frontend

This is where Docker gets most interesting. The admin UI uses a **multi-stage build**:

- **Stage 1 (Node):** Install npm dependencies, run `npm run build` to compile the React app into static HTML/CSS/JS files in a `dist/` folder. The Node.js image with all of `node_modules` is roughly 1 GB.
- **Stage 2 (nginx):** Copy *only* the compiled `dist/` folder into a tiny nginx image (~25 MB). Throw away all of Node.js, npm, and `node_modules`. The result is a production image that is 40x smaller.

Why nginx replaces Vite's dev proxy: During development, when you run `npm run dev`, Vite provides a built-in proxy server that forwards API requests to the FastAPI backend. But in production, Vite is not running — you only have static files. nginx serves those files and also acts as a reverse proxy, forwarding `/auth/`, `/admin/`, and `/health` requests to the search-api container.

Why `env_file: .env` plus a separate `DATABASE_URL` override: The `.env` file contains all your API keys (Groq, Gemini, etc.) and configuration. But it also contains `DATABASE_URL=postgresql://bilingual:bilingual@localhost:5434/bilingual_search` — which is correct for running on your host machine. Inside Docker's network, containers talk to each other by service name, not `localhost`, and postgres listens on its internal port 5432 (the 5434 mapping is only for host access). So we override just `DATABASE_URL` in the environment section with `postgresql://bilingual:bilingual@postgres:5432/bilingual_search` — using Docker's internal DNS name `postgres` and the internal port 5432.

Code walkthrough

1. ETL Dockerfile (`bilingual_etl/Dockerfile`)

A straightforward single-stage Python Dockerfile:

```
# Start from a slim Python 3.10 image (~150 MB instead of ~900 MB for the full image)
FROM python:3.10-slim

# Set the working directory – all subsequent commands run from /app
WORKDIR /app
```

```

# Install system-level dependencies needed by psycopg2 (PostgreSQL driver)
# gcc = C compiler (psycopg2 compiles C extensions during pip install)
# libpq-dev = PostgreSQL client library headers
# --no-install-recommends = skip optional packages, keeps image smaller
# rm -rf /var/lib/apt/lists/* = clean up apt cache, saves ~30 MB
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc libpq-dev && \
    rm -rf /var/lib/apt/lists/*

# Copy requirements.txt FIRST (before source code)
# This layer gets cached – if requirements.txt hasn't changed,
# Docker skips the expensive pip install on subsequent builds
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the ETL package and source data into the image
COPY bilingual_etl/ bilingual_etl/
COPY data/ data/

# Default command: run the ETL pipeline
# The container starts, runs the pipeline, and exits when done
CMD ["python", "-m", "bilingual_etl.scripts.main_etl"]

```

Key pattern: dependency layer caching. Notice that `COPY requirements.txt` and `pip install` come *before* `COPY bilingual_etl/`. If you change your Python code but not your dependencies, Docker reuses the cached pip install layer, and the rebuild only takes seconds instead of minutes.

2. Search API Dockerfile (`search_api/Dockerfile`)

Similar to the ETL Dockerfile, but with one critical difference:

```

# Same slim Python base image
FROM python:3.10-slim

WORKDIR /app

# Same system dependencies for psycopg2
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc libpq-dev && \
    rm -rf /var/lib/apt/lists/*

# Same dependency-first caching pattern
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the search API package
COPY search_api/ search_api/

# ALSO copy bilingual_etl/ – this is the cross-service dependency!
# The search API imports bilingual_etl.providers.embedding_provider
# to generate embeddings for search queries at runtime.
# Without this line, the API would crash with ModuleNotFoundError.
COPY bilingual_etl/ bilingual_etl/

# Document that this service listens on port 8000
# (EXPOSE doesn't actually publish the port – it's documentation
# for humans and tools. The actual port publishing happens in
# docker-compose.yml with the "ports" setting.)
EXPOSE 8000

# Start the FastAPI server
# --host 0.0.0.0 = listen on all interfaces (required inside Docker,
# because "localhost" inside a container only means "this container")
# --port 8000 = match the EXPOSE declaration
CMD ["uvicorn", "search_api.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Why copy both packages? In a monorepo like ours, the search API and the ETL share code. Specifically, when someone searches, the API needs to embed the query text using the same embedding provider the ETL used to embed the documents. The import chain is: `search_api.services -> bilingual_etl.providers.embedding_provider`. If we only copied `search_api/`, Python would not find the `bilingual_etl` module.

3. Admin UI Dockerfile (`admin-ui/Dockerfile`)

A multi-stage build — the most sophisticated of the three:

```
# ===== STAGE 1: Build =====
# Use Node 20 on Alpine Linux (small base) as the build environment
# "AS build" names this stage so we can reference it later
FROM node:20-alpine AS build

WORKDIR /app

# Copy package files FIRST for dependency caching
# (same pattern as requirements.txt in Python)
# npm ci = "clean install" - installs exact versions from lock file,
# faster and more deterministic than npm install
COPY package.json package-lock.json ./
RUN npm ci

# Copy source files and Vite config
COPY index.html vite.config.js ./
COPY src/ src/

# Build the production bundle
# This compiles React JSX, bundles JavaScript, minifies CSS,
# and outputs everything to a /app/dist folder
RUN npm run build

# ===== STAGE 2: Production =====
# Start fresh from a tiny nginx image (~7 MB base)
# Everything from Stage 1 is discarded EXCEPT what we explicitly copy
FROM nginx:alpine

# Copy ONLY the built files from Stage 1
# --from=build references the named stage above
# The dist/ folder contains just HTML, CSS, JS, and assets (~5 MB)
COPY --from=build /app/dist /usr/share/nginx/html

# Copy our custom nginx config (reverse proxy rules)
COPY nginx.conf /etc/nginx/conf.d/default.conf

# nginx listens on port 80
EXPOSE 80

# Start nginx in the foreground (not as a background daemon)
# "daemon off" keeps the process in the foreground so Docker
# can monitor it - if nginx crashes, Docker knows and can restart it
CMD ["nginx", "-g", "daemon off;"]
```

The size difference is dramatic. Stage 1 with Node.js, npm, and all of `node_modules` is roughly 1 GB. Stage 2 with just nginx and the compiled files is roughly 25 MB. That is a 40x reduction. Smaller images mean faster deployments, less storage, and a smaller attack surface.

4. nginx config (`admin-ui/nginx.conf`)

This is the reverse proxy configuration that replaces Vite's dev proxy:

```
server {
```

```
# Listen on port 80 inside the container
# (mapped to port 3000 on the host via docker-compose)
listen 80;

# Serve the React SPA for all frontend routes
location / {
    # Where the static files live (copied from the build stage)
    root /usr/share/nginx/html;
    index index.html;

    # try_files is the key to single-page app routing:
    # 1. Try to serve the exact file requested ($uri)
    # 2. Try it as a directory ($uri/)
    # 3. If neither exists, serve index.html (let React Router handle it)
    # Without this, refreshing on /settings would return a 404
    # because there's no actual /settings file – React Router
    # handles that route client-side
    try_files $uri $uri/ /index.html;
}

# Reverse proxy: forward /auth/ requests to the FastAPI backend
# "search-api" is Docker's internal DNS name for the search-api container
# Docker Compose automatically creates DNS entries for each service name
location /auth/ {
    proxy_pass http://search-api:8000;
}

# Reverse proxy: forward /admin/ requests to the FastAPI backend
location /admin/ {
    proxy_pass http://search-api:8000;
}

# Reverse proxy: forward /health checks to the FastAPI backend
location /health {
    proxy_pass http://search-api:8000;
}
}
```

5. `docker-compose.yml` — the orchestration file

```
services:  
# █ Database ████████████████████████████████████████████████████████  
  
postgres:  
# Use the official pgvector image (PostgreSQL 16 with vector extension)  
# No Dockerfile needed – this image is ready to use from Docker Hub  
image: pgvector/pgvector:pg16  
container_name: bilingual_postgres  
  
# These env vars configure PostgreSQL on first startup  
# (they create the user, password, and database automatically)  
environment:  
    POSTGRES_USER: bilingual  
    POSTGRES_PASSWORD: bilingual  
    POSTGRES_DB: bilingual_search
```

[illegible]

